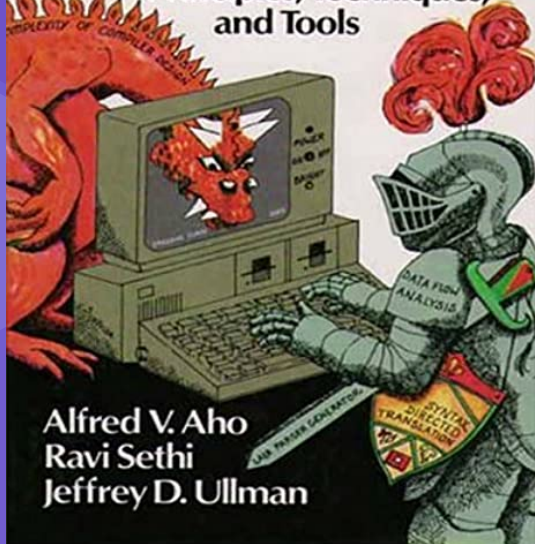


Compilation

Génération, Compilation et Analyse Statique

H. Cassé <hugues.casse@irit.fr>

2022





Organisation

Partie	Enseignant	CM	TD	TP
Compilation	H. Cassé	9h	8h	10h
Méthodes statiques pour le temps-réel	C. Rochange	7h	12h	8h

Compilation :

- génération de code
- analyse statique



Introduction

Pourquoi faire de la compilation ?

- Vous avez **les connaissances** pour faire un compilateur.
- Quand on **conçoit un microprocesseur**, on a besoin d'un compilateur.
- Vous êtes en charge de produire du code : **maîtriser** le compilateur.
- La mise au point peut se faire au niveau **assembleur**.

Objectif du cours :

- Comprendre le fonctionnement d'un compilateur.
- Comprendre les optimisations d'un compilateur.
- Développer des passes de compilation (optimisation, analyse, etc).



Outline

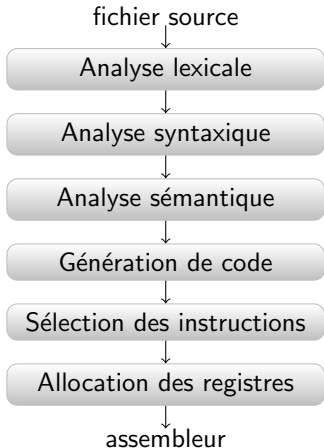
① Introduction

② Allocation de registres

③ Conclusion



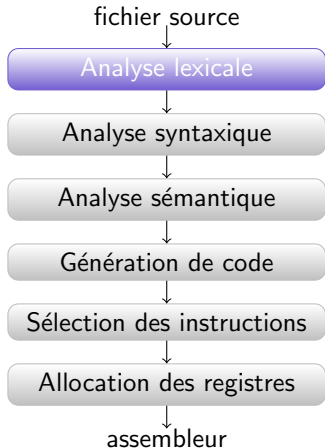
Fichier source



```
int sum(int n) {  
    int s = 0, i = 0;  
    while(i <= n) {  
        s += i;  
        i++;  
    }  
    return s;  
}
```



Analyse lexicale

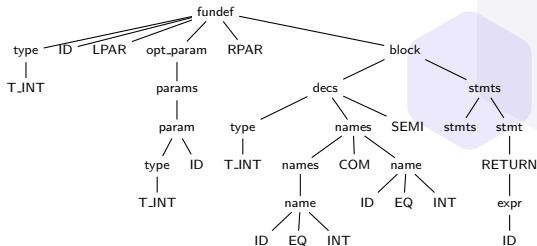
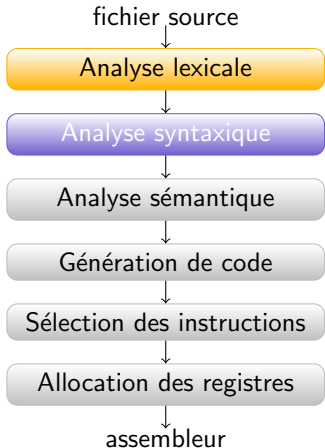


```
[  
  TYPE(T_INT); ID("sum"); LPAREN;  
  TYPE(T_INT); ID("n"); RPAR;  
  LBRACE; TYPE(T_INT); ID("s");  
  EQ; INT(0); COM; ID("i"); EQ;  
  INT(0); SEMI;  
  WHILE; LPAREN; ID("i"); LE;  
  ID("n"); LBRACE; ID("s");  
  PLUS_EQ; ID("i"); SEMI;  
  ID("i"); PLUS_PLUS; RBRACE;  
  RETURN; ID("s"); SEMI;  
  RBRACE  
]
```



Analyse syntaxique

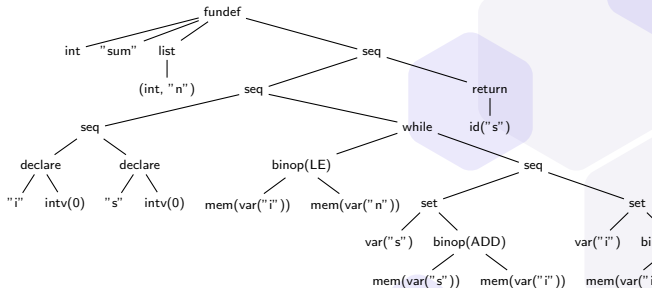
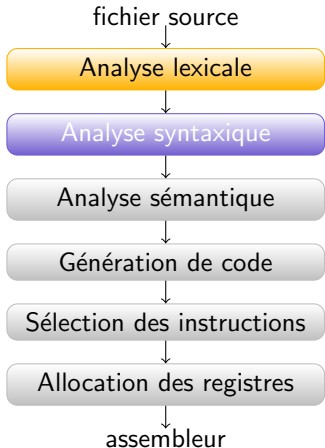
Arbre de dérivation



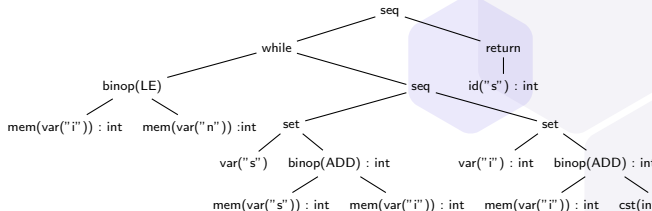
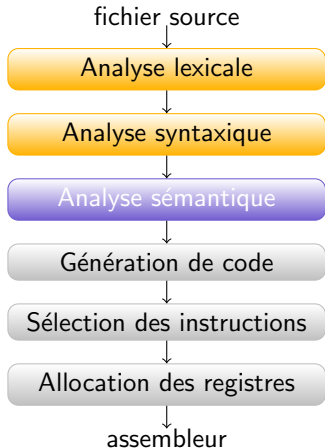


Analyse syntaxique

Arbre de syntaxe abstraite (AST – Abstract Syntactic Tree)



Analyse sémantique



Format des AST

(grammaire abstraite)

$P = D^*$

$D : \text{undef}$

| $\text{vardef}(T, ID)$

| $\text{fundef}(T, ID, (T \times D)^*, S)$

$S : \text{nop}$

| $\text{set}(R, E)$

| $\text{seq}(S, S)$

| $\text{if}(E, S, S)$

| $\text{while}(E, S)$

| $\text{dowhile}(S, E)$

| $\text{call}(ID, E^*)$

| $\text{return}(E)$

| $\text{declare}(ID, T, S)$

$E : \text{none}$

| $\text{cst}(C)$

| $\text{mem}(R)$

| $\text{unop}(\Omega_1, E)$

| $\text{binop}(\Omega_2, E, E)$

| $\text{ecall}(ID, E^*)$

| $\text{cast}(T, E)$

$C : \text{null}$

| $\text{intv}(\mathbb{Z})$

| $\text{floatv}(\mathbb{R})$

$R : \text{noref}$

| $\text{var}(ID)$

$T : \text{void}$

| int

| char

| float

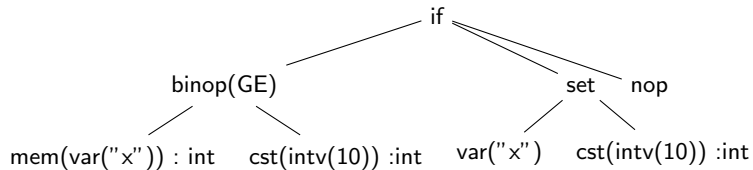
| $\text{ptr}(T)$

$\Omega_1 = \{ \text{NEG, NOT, INV} \}$

$\Omega_2 = \{ \text{ADD, SUB, MUL, DIV, MOD, AND, OR, SHL, SHR, BITAND, BITOR, XOR, EQ, NE, LT, LE, GT, GE} \}$



Notation



```
if(  
  binop(GE,  
    mem(var("x")) : int,  
    cst(intv(10)) : int),  
  set(var("x") : int, cst(intv(10)) : int),  
  nop  
)
```

Valeurs particulières

undef, nop, none, null noref, void – valeurs nulles

call(f, as), ecall(f, as)

- *f* – identificateur de la fonction appelée
- *as* – liste des arguments

*cast(int, x + 1.) – (**int**)(x+1)*

declare(i, int, i = 0; while(i < 10)) –

```
int i = 0;  
while(i < 10) { }
```

seq(i = 0; j = 0) –

```
i = 0;  
j = 0;
```



Exercice 1

Soit les déclarations suivantes :

```
int x, y; float z;
```

Donner l'AST des codes C ci-dessous :

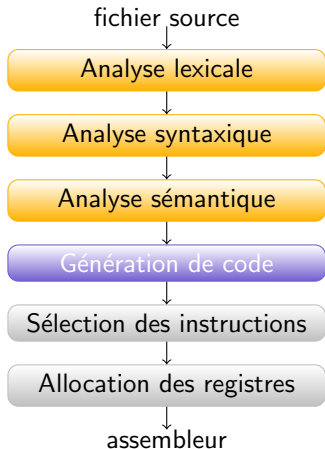
```
/* (a) */ x = x + 1;
/* (b) */ x = ~(x + y);
/* (c) */ x = (int)z;
/* (d) */ z = x;
/* (e) */ return x + 1;
/* (f) */ return;
/* (g) */ x = 0;
           y = 1;
```

```
/* (h) */ if(x > 10)
           y = 0;
           else
           z = 1;
```

```
/* (i) */ if(x > 10)
           y = 0;
           else
           z = 1;
```

```
/* (j) */ for(x = 0; x < 10; x++)
           y = y * 2;
```

Génération de code



label "sum"

$v_i \leftarrow 0$

$v_s \leftarrow 0$

label L₁

goto.ge v_i, v_n, L₂

$v_0 \leftarrow v_s +_{int} v_i$

$v_s \leftarrow v_0$

$v_1 \leftarrow 1$

$v_2 \leftarrow v_i + v_1$

$v_i \leftarrow v_2$

goto L₁

label L₂

return v_s



Langage des quadruplets

$\mathcal{Q}$ ensemble des quadruplets

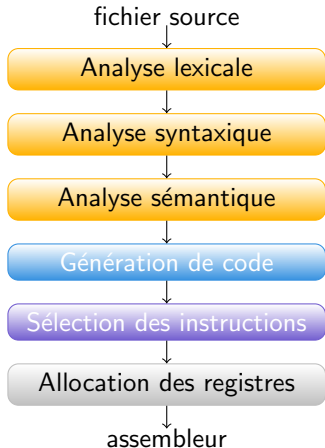
$\mathcal{V}$ registres virtuels (nombre infini)

$\mathcal{L}$ étiquette en assembleur (nombre infini)

$v_i, v_j, v_k \in \mathcal{V}, vs \in \mathcal{V}^*, \tau \in T, c \in \mathbb{Z} \cup \mathbb{R}, \omega_1 \in \Omega_1, \omega_2 \in \Omega_2,$
 $cc \in \{EQ, NE, LT, LE, GT, GE\}, L \in \mathcal{L}$

$v_i \leftarrow c$	label L	call L, vs
$v_i \leftarrow \omega_{1,\tau} v_j$	goto L	return
$v_i \leftarrow v_j \ \omega_{1,\tau} v_k$	goto.cc v_i, v_j, L	push v_i
$v_i \leftarrow M_\tau[v_j]$	$v_i \leftarrow L$	pop v_i
$M_\tau[v_j] \leftarrow v_i$		
$v_i \leftarrow cast_\tau(v_j)$		

Sélection des instructions



sum :

MOV v_i , #0

MOV v_s , #0

L₁ :

CMP v_i , v_n

BGE *L₂*

ADD v_s , v_s , v_i

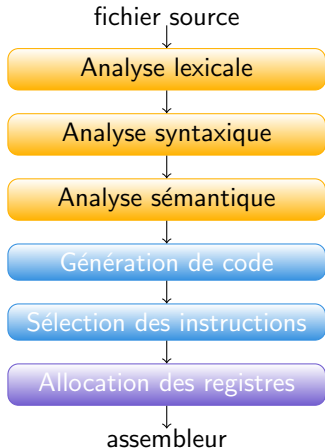
ADD v_i , v_i , #1

B *L₁*

L₂ :

MOV *PC*, *LR*

Allocation des registres



```
/* R0 = n */
```

```
sum :
```

```
    STMFD SP!, [R1 - 2]
```

```
    MOV R1, #0
```

```
    MOV R2, #0
```

```
L1 :
```

```
    CMP R1, R0
```

```
    BGE L2
```

```
    ADD R2, R2, R1
```

```
    ADD R1, R1, #1
```

```
    B L1
```

```
L2 :
```

```
    MOV R0, R2
```

```
    STMFD SP!, [R1 - 2, PC]
```



ABI (Application Binary Interface)

ELF for the ARM Architecture, ABI release 2.09

Utilisation des registres et de la pile :

- PC (R_{15}) – compteur programme
- LR (R_{14}) – registre de lien
- SP (R_{13}) – pointeur de pile
- FP (R_{12}) – pointeur de bloc d'activation
- tout valeur empilée sur 32-bit ou 64-bit

Passage de paramètres et retour de résultat :

- R_0, R_1, R_2, R_3 – 4 premiers paramètres
- autres paramètres passés dans la piles
- la place pour R_{0-3} doit être allouée dans la pile
- R_0 résultat 32-bit, R_{0-1} résultat 64-bit

Conclusion partielle

- 1 La compilation est une suite de traductions.

$$P_1 \in L_1, P_2 \in L_2$$

$$T : L_1 \rightarrow L_2$$

- 2 Comment assurer la validité des transformations ?

$$\mathcal{S} = \mathcal{R} \times \mathcal{M} \rightarrow \mathbb{N}$$

$$P : \mathcal{S} \rightarrow \mathcal{S}$$

Résultat d'un programme : $R : \mathcal{S} \rightarrow \mathbb{N}^n$

$$P_1 = P_2 \iff \forall s \in \mathcal{S}, R(P_1(s)) = R(P_2(s))$$

Solution :

- prouver le compilateur,
- adopter une méthode de codage évitant les bogues.



Outline

① Introduction

② Allocation de registres

③ Conclusion



Plan de travail



Approche :

- sélectionner chaque type d'AST
- examiner sa traduction
- tout ensemble : programme valide



La mémoire

variables locales

- stockées dans registres virtuels
- x locale $\implies v_x \in \mathcal{V}$

variables globales, paramètre

- stockées en mémoire
- x globale $\implies L_x \in \mathcal{L}$

contexte/environnement/table des symboles

$$\begin{aligned} V &\rightarrow \text{undef} \\ &\rightarrow \text{local}(T, \mathcal{V}) \\ &\rightarrow \text{global}(T, \mathcal{L}) \\ \Gamma &= ID \rightarrow V \end{aligned}$$

$$\forall \gamma \in \Gamma, \forall i \in ID,$$

- $\gamma[i]$ – donnée nommé i ,
- $\gamma[i \rightarrow d]$ – nouvelle table où i est associé à d ,
- $\gamma = \lambda i \cdot \text{undef}$ – table vide



Mise en pratique :

traduction des affectations

Soit les définitions suivantes :

```
int x;  
void f(int p) {  
    int i, j;  
    ...  
}
```

Donner l'AST puis réaliser la traduction en quadruplets :

```
1 i = 3;  
2 i = j;  
3 i = x;  
4 x = 0;
```

```
5 i = i + j;  
6 i = i + 1;  
7 x = i + j;  
8 i = (p * x) & ~(1 << (j - 2));
```

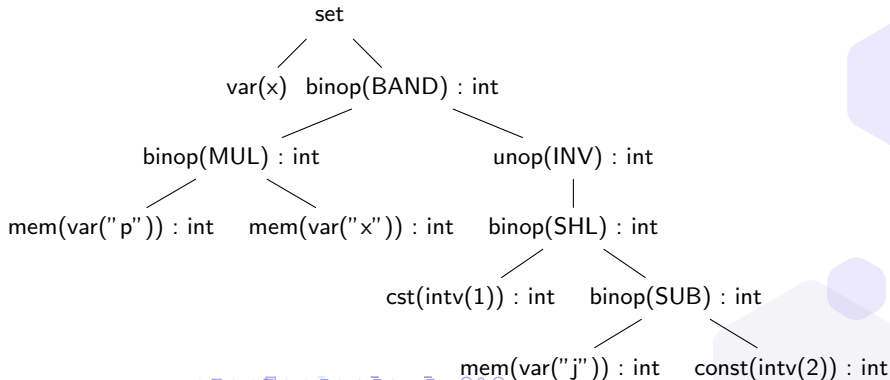
$$\gamma = \left\{ \begin{array}{l} x \mapsto \text{global}(\text{int}, L_x), \\ p \mapsto \text{local}(\text{int}, v_p), \\ i \mapsto \text{local}(\text{int}, v_i), \\ j \mapsto \text{local}(\text{int}, v_j), \end{array} \right\}$$



Étudions ce qu'il se passe...

Consigne :

- Se concentrer sur un nœud à la fois.
- Quelles valeurs transitent entre les nœuds ?
- Dans quel ordre je parcours l'AST ?

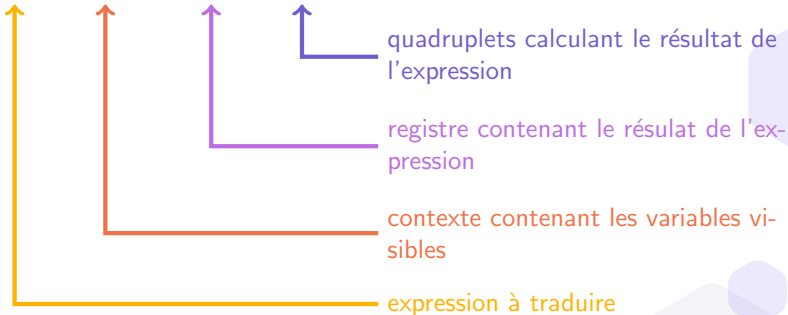




Formalisons...

(focus sur les expressions)

$$[[\cdot]]_E : E \times \Gamma \rightarrow V \times Q^*$$



Écrivons le code.

Génération de registre virtuel : $newtmp : T \rightarrow \mathcal{V}$

$\llbracket const(intv(c)) : \tau \rrbracket_E \gamma = (v, [v \leftarrow c])$
avec $v = newtmp(\tau)$

$\llbracket mem(var(i)) : \tau \rrbracket_E \gamma = ?$
avec $\gamma[i] = local(-, v_i)$ et ...

$\llbracket mem(var(i)) : \tau \rrbracket_E \gamma = ?$
avec $\gamma[i] = global(-, L_i)$ et ...

$\llbracket unop(\omega, e) : \tau \rrbracket_E \gamma = (v, q' @ [v \leftarrow \omega \ v'])$
avec $v = newtmp(\tau)$ et $(v', q') = \llbracket e \rrbracket_E \gamma$.

$\llbracket binop(\omega, e_1, e_2) : \tau \rrbracket_E \gamma = ?$



Affectation

Fonction de traduction des instructions :

$$\llbracket \cdot \rrbracket_S : S \times \Gamma \rightarrow Q^*$$

$$\begin{aligned} \llbracket \text{set}(\text{var}(i), e) \rrbracket_S \gamma = q @ [v_i \leftarrow v] \\ \text{avec } \gamma[i] = \text{local}(_, v_i) \text{ et } (v, q) = \llbracket e \rrbracket_E \gamma. \end{aligned}$$

$$\begin{aligned} \llbracket \text{set}(\text{var}(i), e) \rrbracket_S \gamma = ? \\ \text{avec } \gamma[i] = \text{global}(_, L_i) \text{ et } \dots \end{aligned}$$



Les tableaux

Extension des AST :

- $T_{v2} = T \mid \text{array}(T, \mathbb{N})$
- $R_{v2} = R \mid \text{itemat}(ID, E)$

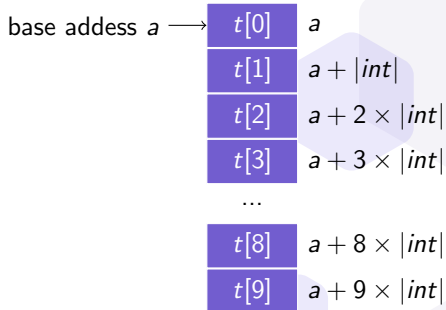
Variable locale tableau : dans la pile,

- $V_{v2} =$
 $\text{undef} \mid \text{global}(T, \mathcal{L}) \mid \text{local}(T, \mathcal{V}, \mathbb{Z})$
- déplacement dans la pile

Syntaxe :

- $t \text{ [...] } \implies \text{mem}(\text{itemat}("t", \dots))$
- $t \text{ [...] } = \dots \implies$
 $\text{set}(\text{itemat}("t", \dots), \dots)$

Implantation en mémoire :
 $\text{array}(\text{int}, 10)$



$$\&t[i] = a + i \times |\text{int}|$$



Générer le code des tableaux

Ecrire les fonctions de traduction suivantes :

- a $\llbracket mem(itemat(i, e)) \rrbracket_E \gamma = ?$
avec $\gamma[i] = global(array(\tau, -), L_i)$ et ...
- b $\llbracket mem(itemat(i, e)) \rrbracket_E \gamma = ?$
avec $\gamma[i] = local(array(\tau, -), -, k_i)$ et ...
- c $\llbracket set(itemat(i, e), e') \rrbracket_S \gamma = ?$
avec $\gamma[i] = global(array(\tau, -), L_i)$ et ...
- d $\llbracket set(itemat(i, e), e') \rrbracket_S \gamma = ?$
avec $\gamma[i] = local(array(\tau, -), -, k_i)$ et ...



Les pointeurs

[t] Au niveau AST :

- $T = \dots \mid ptr(T)$
- $R_{v3} = \dots \mid at(E)$
- $E_{v2} = \dots \mid addr(R)$

Syntaxe :

- $*p \implies mem(at(mem(var(p))))$
- $*p = e; \implies set(at(mem(var(p))), e)$
- $\&x \implies addr(var("x"))$

Écrire les fonctions de traduction :

- a $\llbracket mem(at(e) : ptr(\tau)) : \tau \rrbracket_E \gamma = ?$
- b $\llbracket set(at(e) : ptr(\tau), e') \rrbracket_S \gamma = ?$
- c $\llbracket addr(var(i)) \rrbracket_E \gamma$ avec $\gamma[i] = global(\tau, L_i) ?$
- d $\llbracket addr(var(i)) \rrbracket_E \gamma$ avec $\gamma[i] = local(\tau, v_i, k_i) ?$
- e $\llbracket addr(at(e)) \rrbracket_E \gamma ?$

Toujours simplifier

Revenons sur les pointeurs, essayons de générer :

- $\llbracket \text{binop}(\text{ADD}, e_1 : \text{ptr}(\tau), e_2 : \text{int}) \rrbracket_E \gamma$
- $\llbracket \text{binop}(\text{SUB}, e_1 : \text{ptr}(\tau), e_2 : \text{int}) \rrbracket_E \gamma$
- $\llbracket \text{binop}(\text{SUB}, e_1 : \text{ptr}(\tau), e_2 : \text{ptr}(\tau)) \rrbracket_E \gamma$

A-t-on vraiment besoin de *itemat()* ?

Proposez un autre AST pour :

- $t[i]$
- $t[i] = \dots;$

Les instructions

Rappel : $\llbracket \cdot \rrbracket_S : S \times \Gamma \rightarrow Q^*$

Fonction utile : $newlab : \rightarrow \mathcal{L}$

Générer :

- $\llbracket seq(s_1, s_2) \rrbracket_S \gamma$
- $\llbracket while(c, s) \rrbracket_S \gamma$
- $\llbracket dowhile(s, c) \rrbracket_S \gamma$
- $\llbracket if(c, s_1, s_2) \rrbracket_S \gamma$

avec $c = binop(\omega, e_1, e_2)$ et $\omega \in \mathcal{C} = \{EQ, NE, LT, LE, GT, GE\}$.

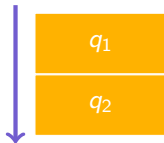
Et les autres conditions ?

- génération avec $\omega \in \{NOT, AND, OR\}$.
- génération avec les autres expressions.



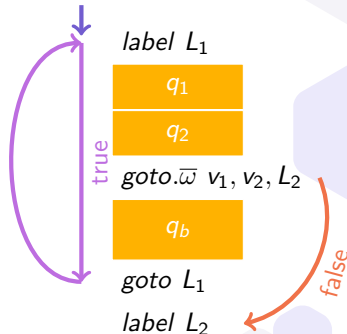
Les schémas de traduction

$\llbracket \text{seq}(s_1, s_2) \rrbracket_s$



avec $q_1 = \llbracket s_1 \rrbracket_s \gamma$ et $q_2 = \llbracket s_2 \rrbracket_s \gamma$.

$\llbracket \text{while}(\text{binop}(\omega, e_1, e_2), s) \rrbracket_s$



avec $q_b = \llbracket s \rrbracket_s \gamma$, $(v_1, q_1) = \llbracket e_1 \rrbracket_E \gamma$ et $(v_2, q_2) = \llbracket e_2 \rrbracket_E \gamma$.

Changer de point de vue

Nouvelle fonction de traduction :

$$\llbracket \cdot \rrbracket_C : E \times L_{true} : \mathcal{L} \times L_{false} : \mathcal{L} \times \Gamma \rightarrow Q^*$$

$\llbracket \cdot \rrbracket_C$ est implanté par :

- $\llbracket binop(\omega, e_1, e_2) \rrbracket_C L_{true} L_{false} \gamma = q_1 @ q_2 @ [goto.\omega \ v_1, v_2, L_{true}; goto \ L_{false}]$
avec $\omega \in \mathcal{C}$, $(v_1, q_1) = \llbracket e_1 \rrbracket_E \gamma$ et $(v_2, q_2) = \llbracket e_2 \rrbracket_E \gamma$
- $\llbracket binop(NOT, e) \rrbracket_C L_{true} L_{false} \gamma ?$
- $\llbracket binop(AND, e_1, e_2) \rrbracket_C L_{true} L_{false} \gamma ?$
- $\llbracket binop(OR, e_1, e_2) \rrbracket_C L_{true} L_{false} \gamma ?$

En utilisant $\llbracket \cdot \rrbracket_C$, re-écrire :

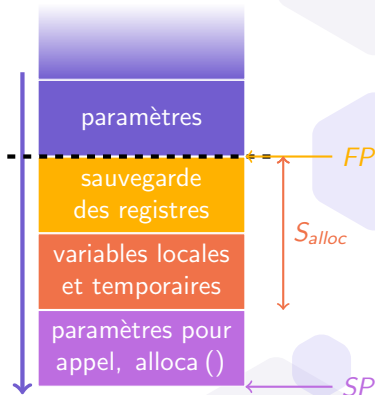
- $\llbracket while(c, s) \rrbracket_S$
- $\llbracket if(c, s_1, s_2) \rrbracket_S$

Le sous-programme/fonction

Bloc d'activation (pile)

- paramètre (de l'appelant)
 k_p – déplacement positif (/ FP)
- registres modifiés + temporaire
(S_{save})
- variables locales + sauvegarde
(S_{loc})
 k_x – déplacement négatif (/ FP)
- partie variable : paramètre pour appel, `alloca()`

$$S_{alloc} = S_{save} + S_{loc}$$



Génération pour les sous-programmes/fonctions

$\llbracket \text{fundef}(t, i, ps, s) \rrbracket_D \gamma =$

Prologue

label L_i

push FP

$FP \leftarrow SP$

push R_i

pour chaque registre modifié

$v_x \leftarrow S_{loc}$

$SP \leftarrow SP - v_x$

$\llbracket s \rrbracket_s \gamma \llbracket ps \rrbracket$

Epilogue

$v_x \leftarrow S_{loc}$

$SP \leftarrow SP - v_x$

pop R_i

pour chaque registre modifié (ordre inverse)

pop FP

return



Appel de sous-programme/fonction

$\llbracket call(i, [e_i]) \rrbracket_s \gamma =$

q_i pour chaque paramètre passé dans un registre
 $R_i \leftarrow v_i$ avec $(v_i, q_i) = \llbracket e_i \rrbracket_E \gamma$
 q_i pour chaque paramètre passé dans la pile (ordre inverse)
 $push\ v_i$ avec $(v_i, q_i) = \llbracket e_i \rrbracket_E \gamma$
 $call\ L_i$



Exercice 2

- a On ajoute les AST *goto*(*ID*) et *label*(*ID*) à *S*. Modifiez *V* (valeurs dans le contexte) et donnez la fonction de génération.
- b On remarque que *return*(*E*) non seulement sélectionne la valeur de retour mais aussi provoque un branchement sur l'épilogue. Donnez la fonction de génération.
- c On ajoute les ASTs *break* et *continue*. Donnez les fonctions de génération.
- d On ajoute les ASTs *switch*(*E*, *S*) et *case*(*C*). Proposez des fonctions de génération.



Outline

① Introduction

② Allocation de registres

③ Conclusion



Exemple

Traduire ce code en assembleur :

```
int sum(int t[], int n) {  
    int s = 0;  
    i = 0;  
    while(i < n) {  
        s = s + t[i];  
        i = i + 1;  
    }  
}
```

$v_s \leftarrow 0$

$v_i \leftarrow 0$

label L_1

goto.GE v_i, v_n, L_2

$v_0 \leftarrow 4$

$v_1 \leftarrow v_i \times v_0$

$v_2 \leftarrow v_t + v_1$

$v_3 \leftarrow M_{int}[v_2]$

$v_4 \leftarrow v_s + v_3$

$v_s \leftarrow v_4$

$v_5 \leftarrow 1$

$v_6 \leftarrow v_i + v_5$

$v_i \leftarrow v_6$

goto L_1

label L_2

$R_0 \leftarrow v_s$

Traduction 1 $\iff$ 1

$v_s \leftarrow 0$

$v_i \leftarrow 0$

label L_1

goto.GE v_i, v_n, L_2

$v_0 \leftarrow 4$

$v_1 \leftarrow v_i \times v_0$

$v_2 \leftarrow v_t + v_1$

$v_3 \leftarrow M_{int}[v_2]$

$v_4 \leftarrow v_s + v_3$

$v_5 \leftarrow v_4$

$v_5 \leftarrow 1$

$v_6 \leftarrow v_i + v_5$

$v_i \leftarrow v_6$

goto L_1

label L_2

$R_0 \leftarrow v_s$

MOV $v_s, \#0$

MOV $v_i, \#0$

L_1 : CMP v_i, v_n

BGE L_2

MOV $v_0, \#4$

MUL v_1, v_i, v_0

ADD v_2, v_t, v_1

LDR $v_3, [v_2]$

ADD v_4, v_s, v_3

MOV v_5, v_3

MOV $v_5, \#1$

ADD v_6, v_i, v_5

MOV v_i, v_6

B L_1

L_2 : MOV R_0, v_s

Instructions utilisées

MOV $v_i, \#c$

MOV v_i, v_j

MUL v_i, v_j, v_k

ADD v_i, v_j, v_k

LDR $v_i, [v_j]$

CMP v_i, v_j

BGE L

B L

Peut-on faire mieux ?

$v_s \leftarrow 0$

$v_i \leftarrow 0$

label L_1

goto.GE v_i, v_n, L_2

$v_0 \leftarrow 4$

$v_1 \leftarrow v_i \times v_0$

$v_2 \leftarrow v_t + v_1$

$v_3 \leftarrow M_{int}[v_2]$

$v_4 \leftarrow v_s + v_3$

$v_5 \leftarrow v_4$

$v_5 \leftarrow 1$

$v_6 \leftarrow v_i + v_5$

$v_i \leftarrow v_6$

goto L_1

label L_2

$R_0 \leftarrow v_s$

MOV $v_s, \#0$

MOV $v_i, \#0$

L_1 : CMP v_i, v_n

BGE L_2

LDR $v_3, [v_t, v_i, LSL \#2]$

ADD v_s, v_s, v_3

ADD $v_i, v_i, \#1$

B L_1

L_2 : MOV R_0, v_s

Instructions utilisées

MOV $v_i, \#c$

MOV v_i, v_j

ADD v_i, v_j, v_k

ADD $v_i, v_j, \#v_k$

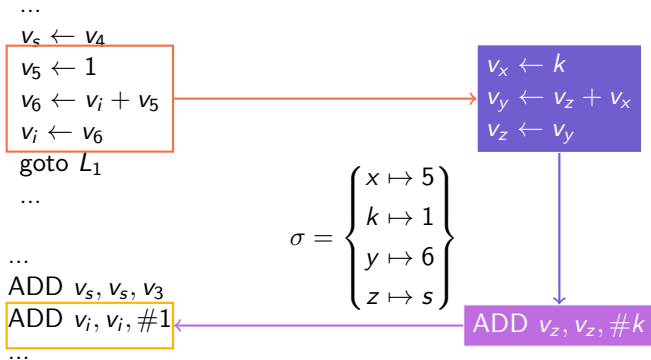
LDR $v_i, [v_j, v_k, LSL \#2]$

CMP v_i, v_j

BGE L

B L

Reconnaître des modèles d'instruction





Est-ce si simple ?

Puis-je remplacer ici ?

```
 $v_x \leftarrow 1$   
label  $L_3$   
 $v_4 \leftarrow v_i + v_x$   
 $v_i \leftarrow v_4$   
 $v_5 \leftarrow 2$   
 $v_6 \leftarrow v_x + v_5$   
 $v_x \leftarrow v_6$   
goto.LT  $v_i, v_n, L_3$ 
```

Règle : le modèle ne s'applique que si les instructions ont une unité d'exécution !



Bloc de base

Definition

Un **Bloc de Base (BB)** est une séquence d'instruction n'acceptant :

- un *label* qu'en première instruction,
- un *goto* qu'en dernière instruction.

Propriété : toutes les instructions d'un BB s'exécutent de manière unitaire.

Graphe de flot de contrôle

Definition

Un **graphe de flot de contrôle** ou **CFG** (*Control Flow Graph*) est un graphe $\langle V, E, \alpha, \omega \rangle$:

V – ensemble des blocs de base,

E – ensemble des arcs $a \rightarrow b$ avec $a, b \in V$,

α, ω sont des BBs vides uniques sans prédécesseur, successeur.

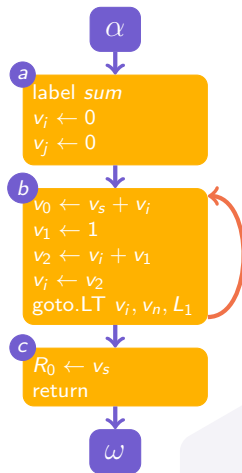
$a \rightarrow b \in E$ ssi :

- a se termine par goto.cc v_i, v_j, L et b commence par label L .
- b suit a dans le code et a ne se termine pas par un branchement.
- b suit a dans le code et a se termine par un branchement conditionnel.



Example

```
label sum  
 $v_i \leftarrow 0$   
 $v_j \leftarrow 0$   
label  $L_1$   
 $v_0 \leftarrow v_s + v_i$   
 $v_1 \leftarrow 1$   
 $v_2 \leftarrow v_i + v_1$   
 $v_i \leftarrow v_2$   
goto.LT  $v_i, v_n, L_1$   
 $R_0 \leftarrow v_s$   
return
```



Exercice

Donner le CFG du programme ci-dessous :

```
int max(int t[], int n) {  
    int m = *t;  
    int i = 1;  
    while(i < n) {  
        int x = t[i];  
        if(x > m)  
            m = x;  
        i++;  
    }  
    return m;  
}
```

```
label sum  
 $v_0 \leftarrow M_{int}[v_t]$   
 $v_i \leftarrow 1$   
label  $L_1$   
 $v_1 \leftarrow 4$   
 $v_2 \leftarrow v_i \times v_1$   
 $v_3 \leftarrow v_t + v_2$   
 $v_x \leftarrow M_{int}[v_3]$   
goto.LE  $v_x, v_m, L_2$   
 $v_m \leftarrow v_x$   
label  $L_2$   
 $v_4 \leftarrow 1$   
 $v_i \leftarrow v_i + v_4$   
goto.LT  $v_i, v_n, L_1$   
 $R_0 \leftarrow v_x$   
return
```



Autre problème

$$v_0 \leftarrow 4$$

$$v_1 \leftarrow v_i \times v_0$$

$$v_2 \leftarrow v_t + v_1$$

$$v_5 \leftarrow 1$$

$$v_3 \leftarrow M_{int}[v_2]$$

$$v_4 \leftarrow v_s + v_3$$

$$v_s \leftarrow v_4$$

$$v_6 \leftarrow v_i + v_5$$

$$v_i \leftarrow v_6$$

goto L_1

$$\begin{aligned} v_x &\leftarrow k \\ v_y &\leftarrow v_z + v_x \\ v_z &\leftarrow v_y \end{aligned}$$

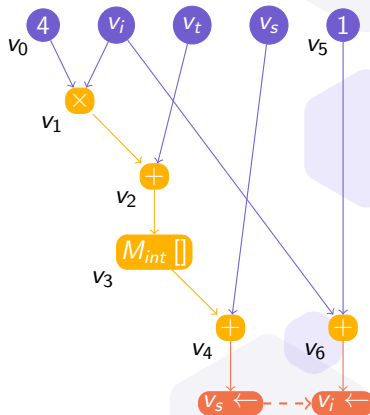
Conclusion : mon système de modèle n'est pas très robuste !



Solution : DAG

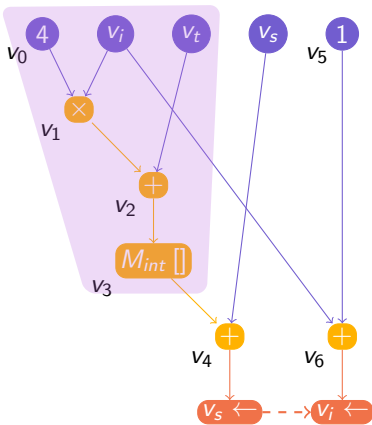
DAG : graphe dirigé acyclique (*Direct Acyclic Graph*)

$v_0 \leftarrow 4$
 $v_1 \leftarrow v_i \times v_0$
 $v_2 \leftarrow v_t + v_1$
 $v_5 \leftarrow 1$
 $v_3 \leftarrow M_{int}[v_2]$
 $v_4 \leftarrow v_s + v_3$
 $v_s \leftarrow v_4$
 $v_6 \leftarrow v_i + v_5$
 $v_i \leftarrow v_6$
goto L_1

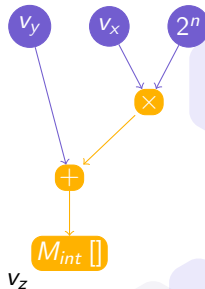




Reconnaissance des modèles



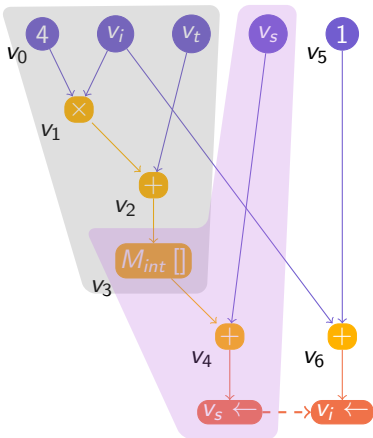
LDR v3, [vt, vi, LSL #2]



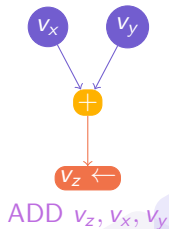
LDR v_z , [v_y , v_x , LSL #c]



Reconnaissance des modèles

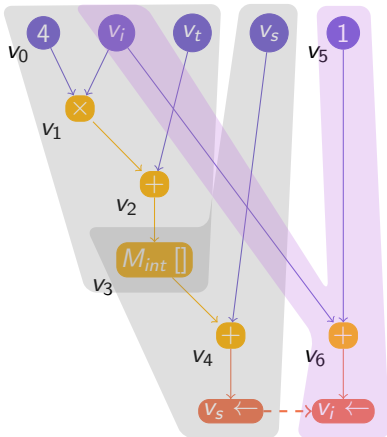


LDR $v_3, [v_t, v_i, \text{LSL} \#2]$
ADD v_s, v_3, v_s

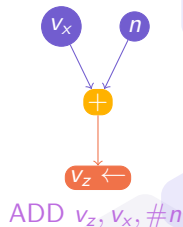




Reconnaissance des modèles



LDR v3, [vt, vi, LSL #2]
ADD v_s, v3, v_s
ADD v_i, v_i, #1



Importance de l'ordre de la mise à jour de l'état

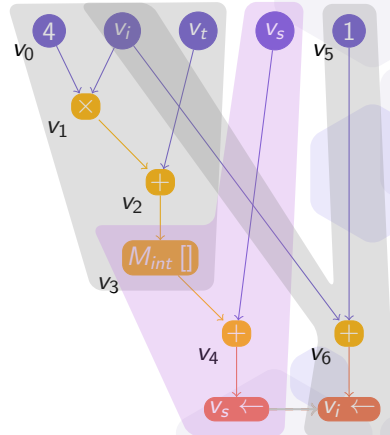
$v_s \leftarrow \dots \rightarrow v_i \leftarrow$

- ordre de sélection des instructions
- identique à celui du programme
- sélectionne la version du v_i

Contre-exemple

```
ADD  $v_i, v_i, \#1$   
LDR  $v_3, [vt, v_i, LSL \#2]$   
ADD  $v_s, v_3, v_s$ 
```

Programme faux : somme des $t[i+1]$





Sélection des instructions

Procédure

- 1 Découper le programme en CFG.
- 2 Pour chaque BB, construire le DAG.
- 3 Reconnaître mes modèles jusqu'à couvrir le DAG.
- 4 Le BB en assembleur est la liste des instructions produites.

= problème de pavage

- problème NP-complet
- en pratique, heuristique linéaire (/ noeuds) $\implies$ 90% de solution optimale



Supporter un jeu d'instruction

Supporter un jeu d'instruction = fournir le modèle de toutes les instructions!

Exemple : support de l'ARM dans LLVM

```
def ADDInst: Instruction {  
  bits<4> rd; bits<4> rn; bits<4> rm;  
  dag OutOperandList = (out IntRegs:$rd);  
  dag InOperandList = (ins IntRegs:$rn, IntRegs:$rm));  
  let AsmString = "add_,$rd,,$rn,,$rm";  
  let Pattern = [(set i32:$rd, (add i32:$rn, i32:$rm))];  
}
```

Exercise

```
int f(int n, int[] t) {  
    int i = -1;  
    int z = 0;  
    while(i < n) {  
        i = i + 1;  
        t[i] = t[i] + 1;  
        if(t[i] == 0)  
            z++;  
    }  
    return z;  
}
```

```
label f  
 $v_i \leftarrow -1$   
 $v_z \leftarrow 0$   
label  $L_1$   
goto.GE  $v_i, v_n, L_2$   
 $v_0 \leftarrow 1$   
 $v_1 \leftarrow v_i + v_0$   
 $v_i \leftarrow v_1$   
 $v_2 \leftarrow 4$   
 $v_3 \leftarrow v_i \times v_2$   
 $v_4 \leftarrow v_t + v_3$   
 $v_5 \leftarrow 4$   
 $v_6 \leftarrow v_i \times v_5$   
 $v_7 \leftarrow v_t + v_6$   
 $v_8 \leftarrow M_{int}[v_7]$   
 $v_9 \leftarrow 1$ 
```

```
 $v_{10} \leftarrow v_8 + v_9$   
 $v_{11} \leftarrow 4$   
 $v_{12} \leftarrow v_i \times v_1$   
 $v_{13} \leftarrow v_t + v_{12}$   
 $v_{14} \leftarrow M_{int}[v_{13}]$   
 $v_{15} \leftarrow 0$   
goto.NE  $v_{14}, v_{15}, L_3$   
 $v_{16} \leftarrow 1$   
 $v_{17} \leftarrow v_z + v_{16}$   
 $v_z \leftarrow v_{17}$   
label  $L_3$   
goto  $L_1$   
label  $L_2$   
 $R_0 \leftarrow v_z$   
return
```



Outline

① Introduction

② Allocation de registres

③ Conclusion



Récapitulatif de la situation

Liste d'instructions

Instruction =

- code assembleur
- bâtie sur des registres virtuels
- ensemble de registres **utilisés** (lus)
- ensemble de registres **mis à jour** (écrits)

Renommage des registres

- ensemble infini de registres virtuels ($\mathcal{V}$)
- ensemble fini de registres matériels ($\mathcal{R}$)
- on cherche une table de substitution : $\sigma : \mathcal{V} \rightarrow \mathcal{R}$

Propriété : si σ est une substitution valide

alors $\sigma[v_i \mapsto \sigma[v_j], v_j \mapsto \sigma[v_i]]$ est aussi valide !



Pas assez de registres ?

Variable – à chaque point du programme

- dans un registre
- en mémoire

Cas de l'ARM

- 4 premiers paramètres dans registres R_{0-3}
- paramètres suivants dans la pile
- variables globales en mémoire

Pas assez de registres :

- stocker dans la pile (offset k_x)
- registre dédié R_{tmp}
- avant utilisation – LDR $R_{tmp}, [FP, \#k_x]$
- après mise à jour – STR $R_{tmp}, [FP, \#k_x]$

Au niveau bloc de base

Procédure

Contrainte : entre les blocs, toutes les données sont stockées en mémoire !

- 1 Tous les registres utilisés sont chargés en début de BB.
- 2 Tous les registres mis à jour sont sauvegardés en fin de BB.

```
LDR v3, [vt, vi, LSL #2]  
ADD vs, vs, v3  
ADD vi, vi, #1
```

$$\sigma = \left\{ \begin{array}{l} v_3 \mapsto R_0, \\ v_t \mapsto R_1, \\ v_i \mapsto R_2, \\ v_s \mapsto R_3 \end{array} \right\}$$

```
LDR R0, [FP, #k3]  
LDR R1, [FP, #kt]  
LDR R2, [FP, #ki]  
LDR R3, [FP, #ks]  
LDR R0, [R1, R2, LSL #2]  
ADD R3, R3, R0  
ADD R2, R2, #1  
STR R0, [FP, #k3]  
STR R2, [FP, #ki]  
STR R3, [FP, #ks]
```

Vivacité des registres

Definition

En un point du program $L \in \mathcal{L}$, un registre virtuel $v \in \mathcal{V}$ est vivant ssi (a) il a été mis à jour précédemment et (b) il va être utilisé par la suite.

```
0 : LDR v3, [v_t, v_i, LSL #2]
1 : ADD v_s, v_s, v3
2 : ADD v_i, v_i, #1
```

v_i, v_s : vivant avant et après le BB

v_t : vivant seulement jusqu'à 0

v_3 : vivant entre 0 et 1

```
LDR R1, [FP, #k_t]
LDR R2, [FP, #k_i]
LDR R3, [FP, #k_s]
LDR R0, [R1, R2, LSL #2]
ADD R3, R3, R0
ADD R2, R2, #1
STR R2, [FP, #k_i]
STR R3, [FP, #k_s]
```



Exercice

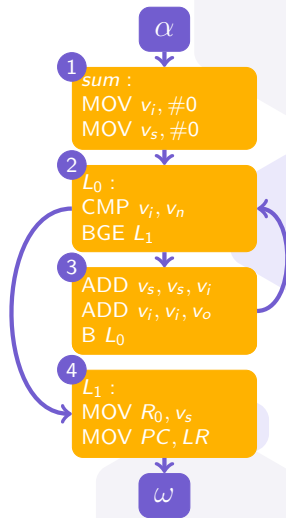
En profitant de la vivacité des registres, faites l'allocation des registres de la séquence ci-dessous :

```
0 : LDR v3, [vt, vi, LSL #2]
1 : ADD v4, vi, #1
2 : LDR v5, [vt, v4, LSL #2]
3 : ADD v6, v3, v5
4 : STR v6, [vt, vi, LSL #2]
5 : ADD vi, vi, #1
```

```
LDR R0, [FP, #ki]
LDR R1, [FP, #kt]
0 : LDR R2, [R1, R0, LSL #2]
1 : ADD R3, R0, #1
2 : LDR R3, [R1, R3, LSL #2]
3 : ADD R3, R2, R3
4 : STR R3, [R1, R0, LSL #2]
5 : ADD R0, R0, #1
STR R0, [FP, #kj]
```

Remontons au niveau CFG

- 1 Dans le 1er BB, sauvegardez les paramètres v_t , v_n et v_o .
- 2 Appliquez la méthode aux BB du CFG ci-contre.
- 3 Que constatez vous ?
- 4 Comment améliorer la situation ?
- 5 Procédez à l'allocation des registres de manière globale.
- 6 Remarquez que si v_i est affecté à un bon registre, on peut éliminer une instruction. Quel registre ?

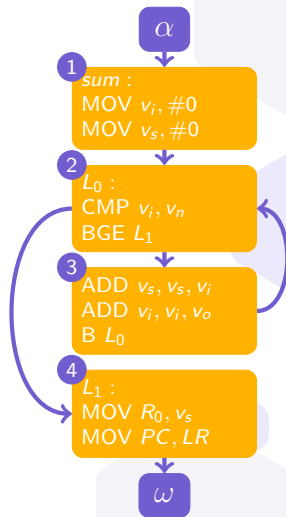


Comment choisir les registres globalement ?

On décide de n'allouer globalement que 2 registres R_0 et R_1 .

Pour les autres variables, on applique l'algorithme au niveau BB.

- 1 Quels registres virtuel affecte-t-on à R_0 et R_1 ?
- 2 Faites l'allocation de registres avec cette affectation.



Allocation globale

$\mathcal{N}$ premiers registres de $\mathcal{R}$

Coût hors registre pour v_x :

$$C_{v_i} = \left(\sum_{x \in USE_{v_i}} 10^{|L_x|} C_{load} \right) + \left(\sum_{x \in UPD_{v_i}} 10^{|L_x|} C_{store} \right)$$

- coût de chargement pour utiliser $v_x - C_{load}$ (5 par exemple),
- coût de stockage pour mettre à jour $v_x - C_{store}$ (2 par exemple),
- $USE_{v_i} \subset V$ – BB utilisant v_i ,
- $UPD_{v_i} \subset V$ – BB mettant à jour v_i ,
- $L_x \subset V$ – ensemble des boucles contenant L_x
- BB dans une boucle s'exécute plusieurs fois (heuristique – 10 fois par boucle).

Allouons à des registres globaux les variables les plus coûteuses !

Question : Comment choisir $\mathcal{N}$?

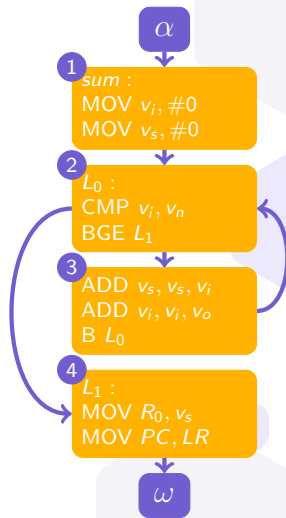
Exercice

Appliquez l'approche globale au CFG ci-contre avec $\mathcal{N} = 2$.

- 1 Quelles variables sont allouées aux registres globaux ?
- 2 Faîtes l'allocation des registres.

On pourra s'aider du tableau ci-dessous :

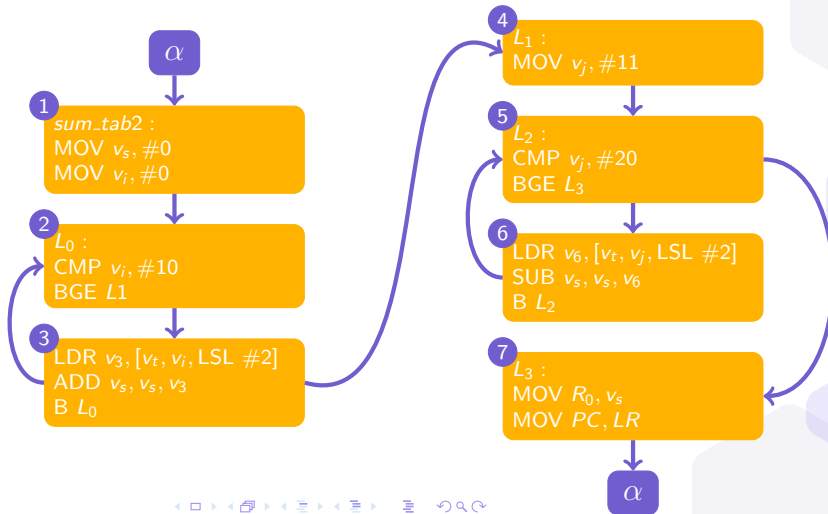
Variable	USE_v	UPD_v	C_v^{USE}	C_v^{UPD}	C_v
v_i					
v_s					
...					





Exemple pathologique

LE CFG



Exemple pathologique

Calcul des coûts

Variable	USE_v	UPD_v	C_v^{USE}	C_v^{UPD}	C_v
v_i	$\{ 2, 3 \}$	$\{ 1, 3 \}$	100	22	122
v_j	$\{ 5, 6 \}$	$\{ 4, 5 \}$	100	22	122
v_s	$\{ 3, 6 \}$	$\{ 1, 3, 6 \}$	100	22	122
v_t	$\{ 3, 6 \}$	$\emptyset$	100	0	100
v_3	$\{ 3 \}$	$\{ 3 \}$	50	10	60
v_7	$\{ 6 \}$	$\{ 6 \}$	50	10	60

$$\mathcal{N} = 2$$

$$\sigma = \left\{ \begin{array}{l} v_i \mapsto R_0 \\ v_j \mapsto R_1 \\ v_s \mapsto \text{pile!} \end{array} \right\}$$

Mais : v_i et v_j ne sont jamais vivants en même temps dans le CFG !

$$\sigma = \left\{ \begin{array}{l} v_i, v_j \mapsto R_0 \\ v_s \mapsto R_1 \end{array} \right\}$$



Graphe d'interférence

Définition

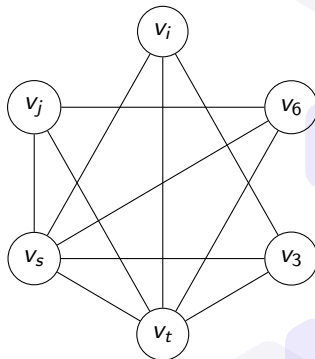
Graphe non-orienté :

$$G_{int} = \langle V_{inter}, E_{inter} \rangle$$

- $V_{int} = \mathcal{V}$
- $E_{int} \subset V_{int} \times V_{int}$

Definition

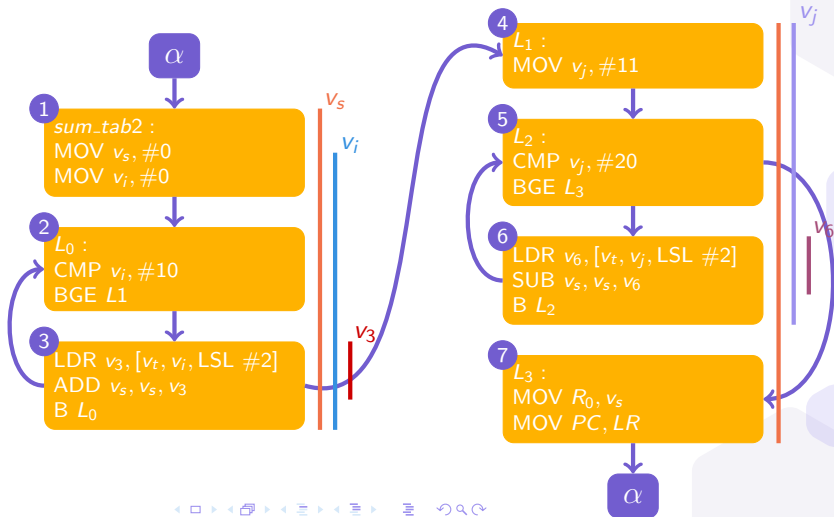
$(v_x, v_y) \in E_{int}$ ssi il existe un point du CFG où v_x et v_y sont vivants en même temps !





Graphe d'interférence

Durée de vie des registres virtuels





Problème de coloriage

Règle : si $(v_x, v_y) \in E_{inter}$ alors v_x et v_y ne peuvent pas être affecté au même registre matériel !

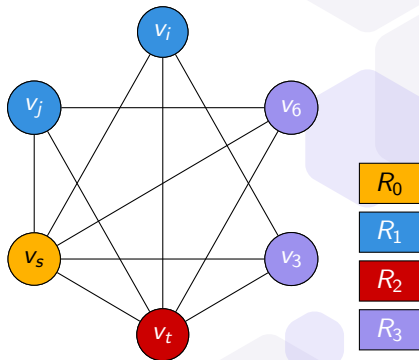
Type de problème : coloriage de graphe.

1 registre matériel = 1 couleur

Complexité : NP-complet

Mais...

- R_{tmp} si coloriage impossible
- heuristique marchant dans 90% des cas





Allocation par coloriage

Idée : construire une liste/ordre de coloriage.

Règle 1 : tout registre ayant moins de $|\mathcal{R}|$ voisin est toujours coloriable
 $\implies$ l'enlever du graphe ne change pas le problème.

Règle 2 : parmi les registres virtuels en conflit
 $\implies$ choisir celui qui est le moins intéressant à colorier.

Heuristique :

- ◆ le moins coûteux – optimisation temporelle
- ◆ le plus de conflit – maximum de déblocage

$$W_{v_x} = \frac{C_{v_x}}{|v_x|}$$

Sélectionner le W_{v_x} le plus petit.



Calcul des W_{v_x} et ordonnancement

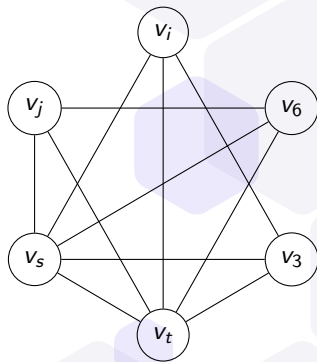
Var.	USE_v	UPD_v	C_v^{USE}	C_v^{UPD}	C_v	$ v_x $	W_{v_x}
v_i	{ 2, 3 }	{ 1, 3 }	100	22	122	3	40
v_j	{ 5, 6 }	{ 4, 5 }	100	22	122	3	40
v_s	{ 3, 6 }	{ 1, 3, 6 }	100	22	122	5	54
v_t	{ 3, 6 }	$\emptyset$	100	0	100	4	20
v_3	{ 3 }	{ 3 }	50	10	60	3	20
v_6	{ 6 }	{ 6 }	50	10	60	3	20

$|\mathcal{R}| = 4$

Ordre : $v_s, v_i, v_j, v_t, v_3, v_6$.

Sélection : ordre inverse, pile.

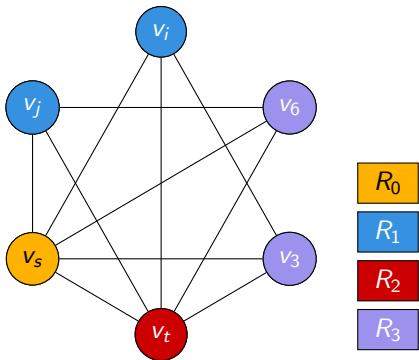
Convention : pour standardiser les corrections, on prend le W_{v_x} le plus petit, sinon le plus grand chiffre, sinon la plus grande lettre.





Coloriage

Ordre : $v_s, v_i, v_j, v_t, v_3, v_6$



$$\sigma = \left\{ \begin{array}{l} v_s \mapsto R_0 \\ v_i \mapsto R_1 \\ v_j \mapsto R_1 \\ v_t \mapsto R_2 \\ v_3 \mapsto R_3 \\ v_6 \mapsto R_3 \end{array} \right\}$$



Après allocation des registres

sum_tab2 :

MOV R_0 , #0

MOV R_1 , #0 $L_0 :$

CMP R_1 , #10

BGE L_1

LDR R_3 , [R_2 , R_1 , LSL #2]

ADD R_0 , R_0 , R_3

B L_0

$L_1 :$

MOV R_1 , #11

$L_2 :$

CMP R_1 , #20

BGE L_3

LDR R_3 , [R_2 , R_1 , LSL #2]

SUB R_0 , R_0 , R_3

B L_2 $L_3 :$

MOV R_0 , R_0

MOV PC, LR

Exercice

$$|\mathcal{R}| = 3$$

- 1 Construire le CFG.
- 2 En déduire le graphe d'interférence.
- 3 Construire la table des poids.
- 4 En déduire l'ordre d'allocation.
- 5 Faire l'allocation des registres.
- 6 Re-écrire le code avec les registres alloués.

max :

MOV $v_x, \#0$

MOV $v_i, \#0$

*L*₀ :

CMP v_i, v_n

BGE *L*₂

LDR $v_3, [v_t, v_i, \text{LSL} \#2]$

CMP v_x, v_3

BGE *L*₁

MOV v_x, v_3

*L*₁ :

ADD $v_i, v_i, \#1$

B *L*₀

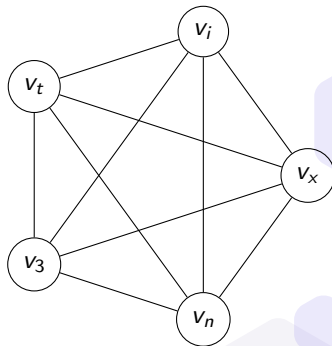
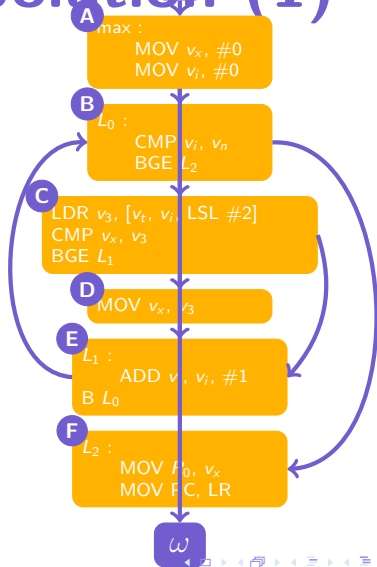
*L*₂ :

MOV R_0, v_x

MOV *PC*, *LR*



Solution α (1)





Solution (2)

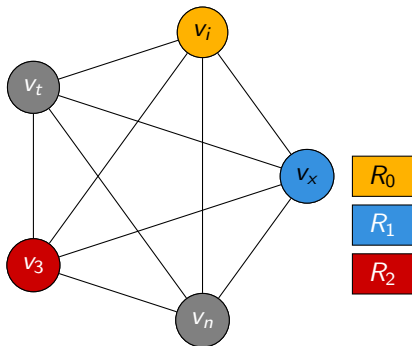
Var.	USE_v	UPD_v	C_v^{USE}	C_v^{UPD}	C_v	$ v_x $	W_{v_x}
v_i	B, C, E	A, E	150	22	172	5	34
v_n	B		50	0	50	5	10
v_t	C		50	0	50	5	10
v_x	C, F	A, D	55	22	77	5	15
v_3	C, D	C	100	20	70	5	14

Ordre : v_i , v_x , v_3 , v_n , v_t



Solution (3)

Ordre : v_i , v_x , v_3 , v_n , v_t



max :

MOV R_1 , #0

MOV R_0 , #0

L_0 :

LDR R_{tmp} , [FP, # k_n]

CMP R_0 , R_{tmp}

BGE L_2

LDR R_{tmp} , [FP, # k_t]

LDR R_2 , [R_{tmp} , R_0 , LSL #2]

CMP R_1 , R_2

BGE L_1

MOV R_1 , R_2

L_1 :

ADD R_0 , R_0 , #1

B L_0

L_2 :

MOV R_0 , R_1

MOV PC, LR



Outline

① Introduction

② Allocation de registres

③ Conclusion



Conclusion

Ce qu'il faut retenir :

- ◆ La compilation est une suite de traductions.
- ◆ Validité $\implies$ approche formelle.
- ◆ Autant d'étapes que nécessaires pour simplifier les problèmes.
- ◆ Identifier problèmes complexes $\implies$ différentes solutions développées séparément.

Transformations

- ◆ applicable sur AST, quadruplets, assembleur.
- ◆ bon exemple : allocation de registre
 - 1 analyse $\implies$ vivacité, graphe d'interférence, coloriage
 - 2 remplacement des registres virtuels par des matériels

Attention : Préserver la sémantique des programmes !

Pour approfondir ...

Comment traduire ...

- que faire si des instructions ne sont pas codables (immédiat trop long dans $\text{MOV } R_i, \#k$) ?
- des opérations non disponibles sur l'architecture (division sur ARM) ?
- la structure des objets des langages objet ?
- la résolution dynamique des méthodes ?
- le mécanisme des exceptions ?

Comment profiter ...

- des instructions conditionnelles (tel que ARM) ?
- des instructions vectorielles de certains processeur ?
- du parallélisme disponible (multicoeurs, GPU) ?